



BITS Pilani

Cloud Computing

SEWP ZG527

Agenda

innovate

achieve

lead

01

Cloud Recap

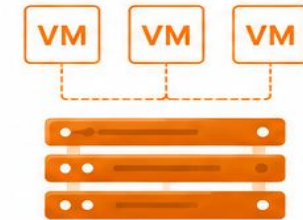
- ◆ What is NIST 3-4-5 Rule
- ◆ Advantages of Cloud
- ◆ Disadvantages



04

Early Approaches to Virtualization

- ◆ Emulation
- ◆ Binary Translation
- ◆ Para Virtualization
- ◆ Hardware Assisted Virtualization
- ◆ Single Kernel



02

Introduction to Virtualization

- ◆ Virtual box Demo
- ◆ History & Evolution of Virtualization
- ◆ Need for Virtualization



05

Applications of Virtualization

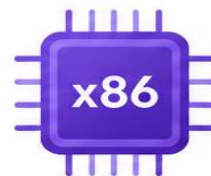
- ◆ Server Virtualization
- ◆ Storage Virtualization
- ◆ Memory Virtualization
- ◆ Network Virtualization
- ◆ Device Virtualization



03

The x-86 Architecture

- ◆ What is the x-86 Architecture
- ◆ Challenges to Virtualising x-86
- ◆ Solutions



06

Conclusion

- ◆ Pro & Cons of Virtualization
- ◆ Future of Virtualization





BITS Pilani

Pilani | Dubai | Goa | Hyderabad



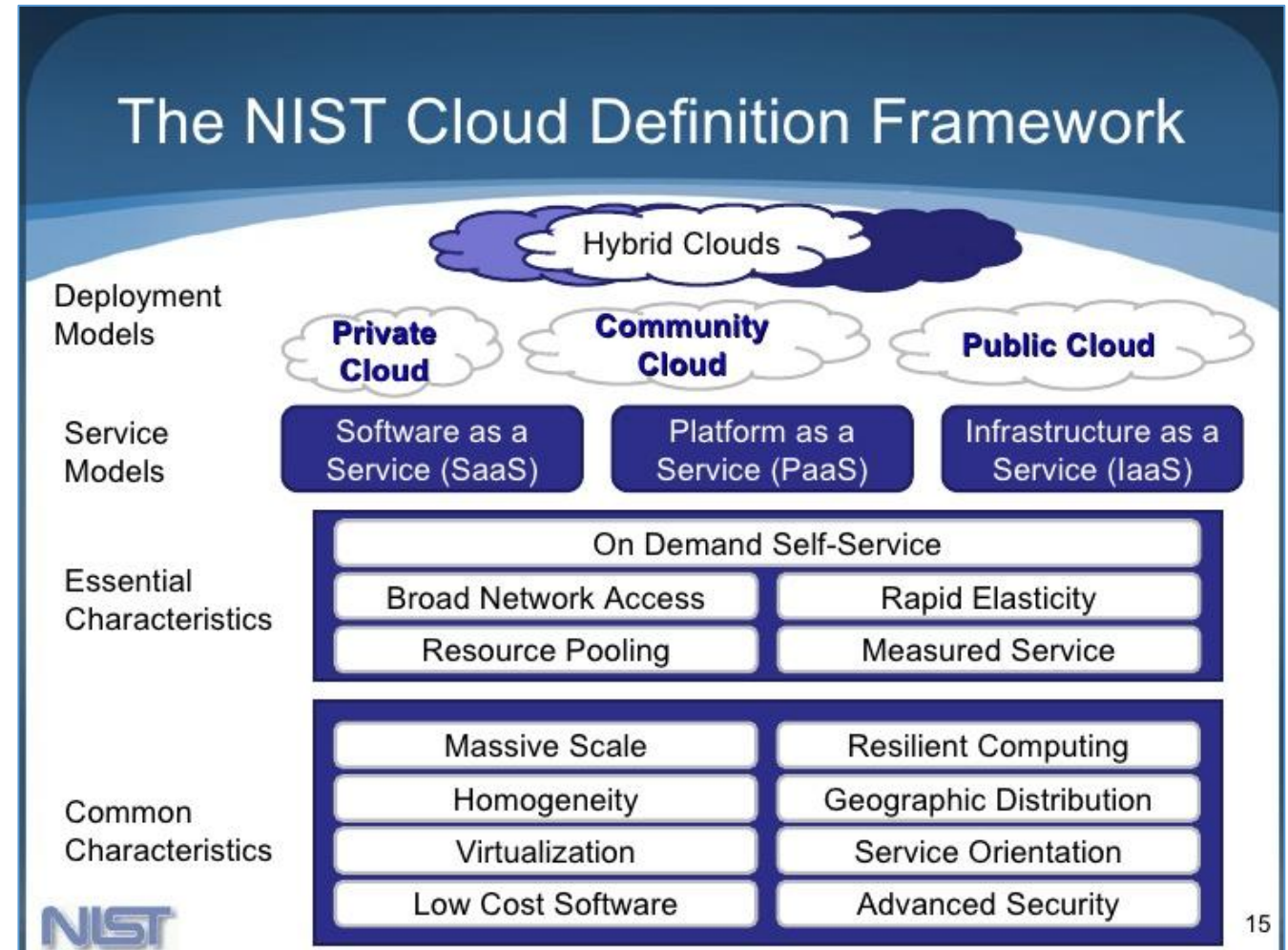
Recap



NIST Definitions



- **3** cloud service models or service types for any cloud platform
- **4** Deployment models
- **5** Essential characteristics of cloud computing infrastructure

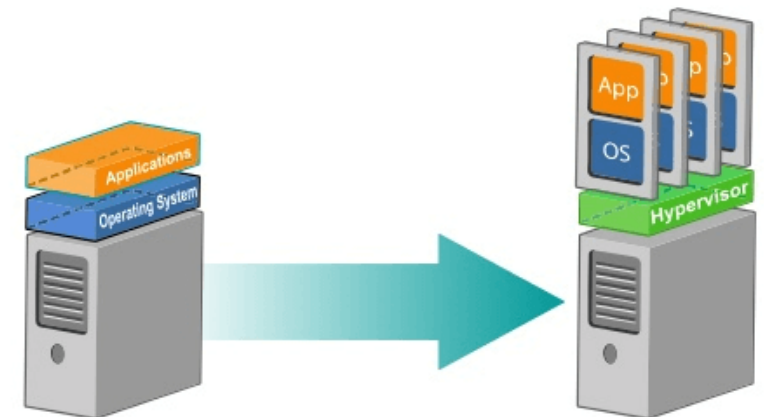




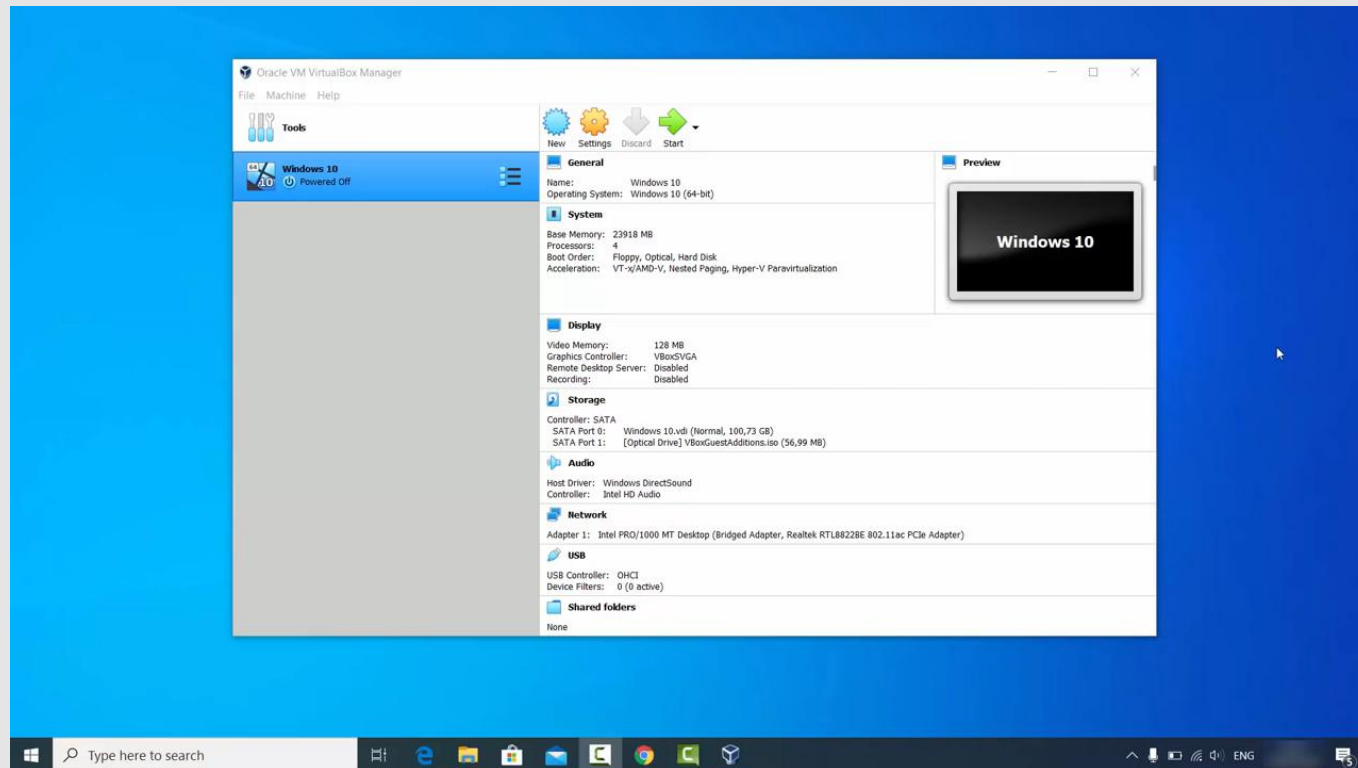
BITS Pilani

Pilani | Dubai | Goa | Hyderabad

Introduction to Virtualization



Video – Virtualization



Learning Objectives

- Introduce **Oracle Virtual Box**, a hosted hypervisor.
- Demonstrate what a **host system** is what a **guest VM** is and what is the role of the **hypervisor**.
- Students will use the same as **home work** and install virtual box and a choice of their own OS after class.

New Terms to Remember

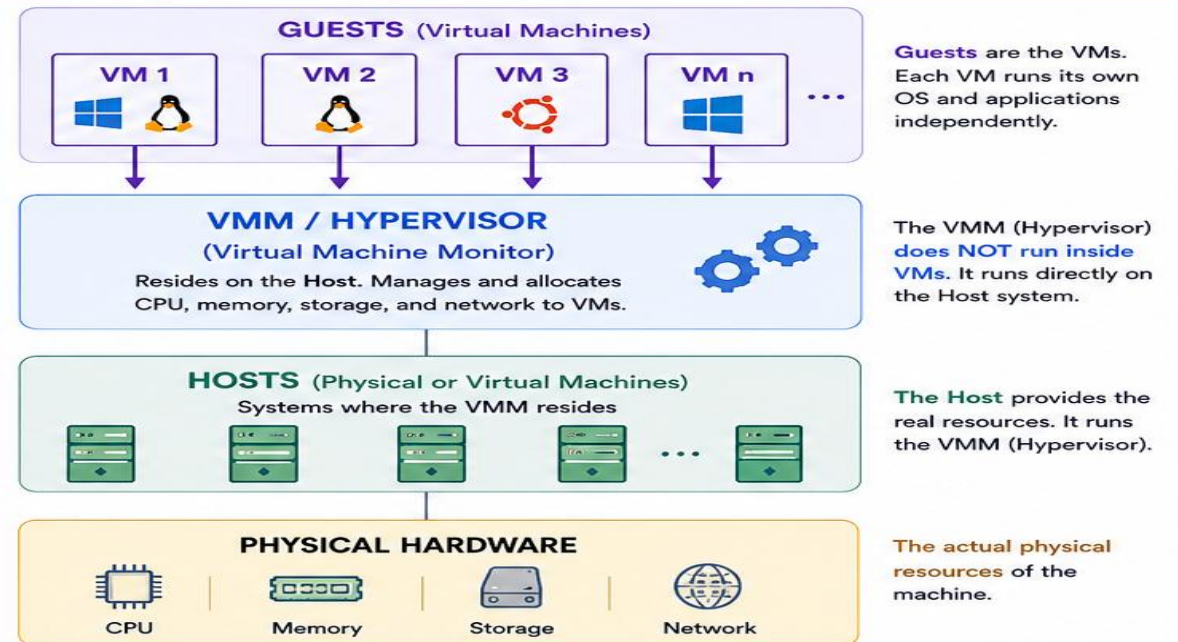
KEY TERMS IN VIRTUALIZATION

Remember all 7 key terms with this complete mnemonic and visual!

MNEMONIC: “**MY** **VERY** **HAPPY** **MANY** **HOSTS** **RUN** **GUESTS**”

M	My	→	VM Virtual Machine	A virtual computer created to run an OS and applications.	
V	Very	→	VMM Virtual Machine Monitor	The software layer that manages and controls VMs.	
H	Happy	→	Hypervisor = VMM Another name for the Virtual Machine Monitor	It creates, runs and manages VMs and allocates resources.	
M	Many	→	Multiplexed Many or several	The VMM multiplexes (shared use of) the hardware among many VMs.	
H	Hosts	→	Host System where the VMM resides	The physical (or virtual) machine that runs the hypervisor.	
R	Run	→	Runs Supports and executes	The host runs the VMM, which runs the VMs and their workloads.	
G	Guests	→	Guest Virtual Machines created	The VMs are called guests because they run on the host.	

THE BIG PICTURE: WHERE EVERYTHING LIVES



QUICK MEMORY TIPS



A VM is a virtual computer inside the system.



VMM (Hypervisor) is the manager of all VMs.



Multiplexed means sharing one host's resources among many VMs.



Hosts are the real systems that host the VMM.



Runs means the host runs the VMM, which runs the VMs.



Guests are the VMs that run on the host, managed by the VMM.



Hypervisor and VMM are the same thing!

★ Think: **My Very Happy Many Hosts Run Guests** — remember all 7 key terms!

Virtualization History

The Origin of Virtualization

From a Single Machine to Limitless Possibilities

EARLY 1960s

01 One Computer, Many Minds



In a large scientific institute, there was only one computer for hundreds of employees.

Scientists wrote programs and submitted them to an operator who ran them and returned the results.

THE CHALLENGE

02 Computing Was Rare, Slow & Expensive



Computers were scarce, slow and extremely expensive.

Direct access for programmers was inefficient—while one person entered data or paused to think, the machine sat idle. Idle time was unacceptable, so developers and computers were separated.

THE BREAKTHROUGH

03 The Birth of Time-Sharing

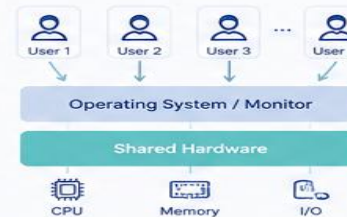


Scientists realized that if more than one user could use the computer, efficiency would increase.

While one user enters data, the computer works on another user's task—filling pauses and minimizing idle time. This was the birth of the **Time-sharing concept**.

THE IMPACT

04 Time-Sharing Led to Virtualization



Time-sharing enabled multiple users to share computing resources concurrently.

This revolutionary idea paved the way for **virtualization**—abstracting resources to deliver **more, to more users, more efficiently**.

TODAY

05 From One Machine to Limitless Possibilities



Virtualization is now the foundation of modern computing.

It powers cloud platforms, data centers, containers, and digital transformation—enabling scalability, agility and innovation at global scale.

One idea. Endless impact.



The Time-sharing Concept

Computing resources are shared among many users.

It appeared in the early 1960s and led to many revolutionary changes, including the emergence of virtualization.



Single User
(Serial Processing)



Time-Sharing
(Multi-User Access)

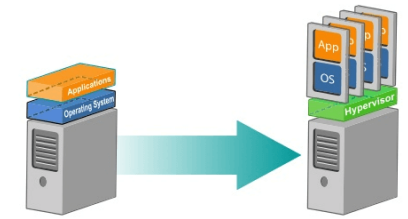


Virtualization
(Abstract Resources)



Cloud & Beyond
(Elastic, On-Demand)

What is Virtualization?



What is Virtualization?

One Physical Machine (Host). Multiple Virtual Worlds (Guests).



Virtualization is a computer architecture technology that allows **multiple virtual machines (VMs)** to be multiplexed on the same physical hardware (**Host**).



Run Multiple Operating Systems (Guests)
Virtualization enables multiple operating system instances to run **concurrently on a single computer (Host)**, isolated from each other.

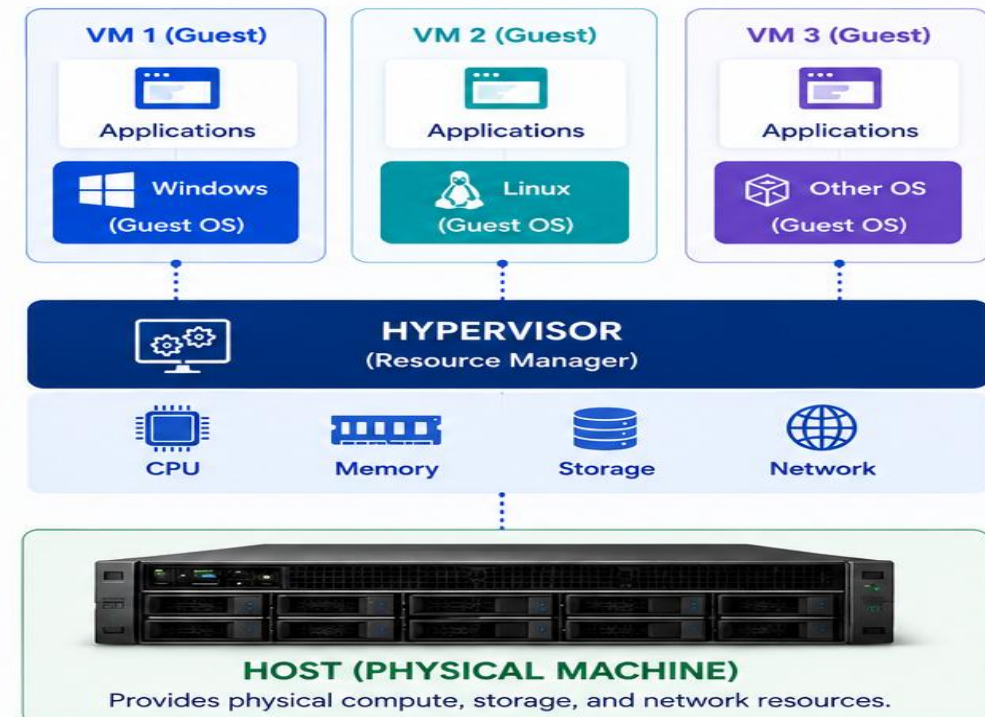


Maximize Hardware Efficiency (Host)
Instead of buying and maintaining separate hardware for each application, each one gets its own operating system while **sharing the same physical resources**.



Intelligent Resource Management (Host)
The hypervisor (or operator) manages and allocates CPU, memory, storage, and other resources to each VM (Guest), ensuring **every guest gets what it needs**—nothing more, nothing less.

MULTIPLE VIRTUAL MACHINES (GUESTS)



Virtualization abstracts hardware to create isolated, secure, and efficient environments, enabling better utilization, agility, scalability, and cost savings.



Better Utilization



Lower Costs



High Availability



Scalability & Agility

What is Virtualization?

Objectives of Virtualization

Do more with less. Run more, use better, deliver faster.

HIGH LEVEL OVERVIEW

Virtualization abstracts computing resources from physical hardware to create **multiple isolated virtual environments**. It enables optimal utilization of hardware, flexibility in managing workloads, and simplicity in operations.



Maximize Resource Utilization

Consolidate workloads on fewer physical machines and make the best use of CPU, memory, storage and network.



Reduce Costs

Lower capital and operational expenses through consolidation, reduced power, cooling, and hardware footprint.



Increase Flexibility & Agility

Rapidly provision, clone, move or scale virtual machines to respond to business needs quickly.



Improve Availability & Resilience

Support high availability, fault tolerance and workload mobility to minimize downtime and ensure business continuity.



Simplify Management

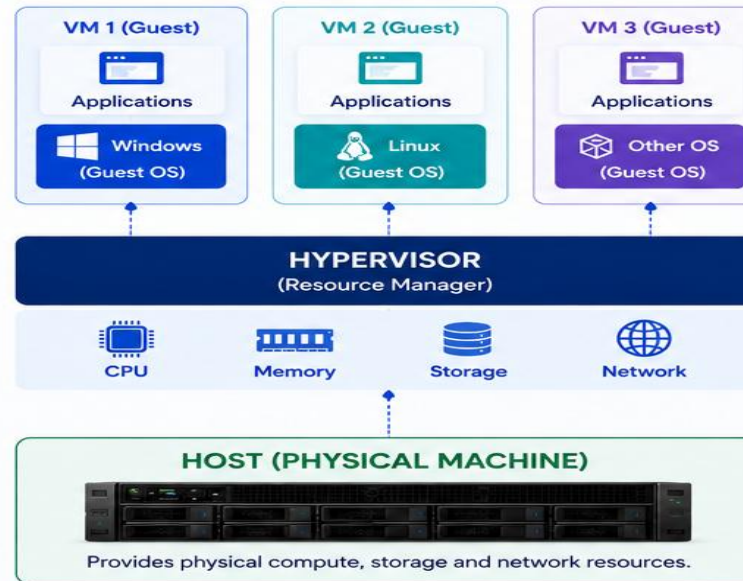
Centralize management of infrastructure and workloads, automate operations and reduce complexity.



CORE PURPOSE

To abstract and pool physical resources so multiple isolated environments (Guests) can run efficiently on a single physical machine (Host) while ensuring security, performance and control.

HOW IT WORKS (AT A HIGH LEVEL)



WHAT IT ENABLES



Run Multiple Workloads

Run multiple operating systems and applications simultaneously on the same hardware.



Isolation & Security

Each virtual machine is isolated from others, enhancing security and stability.



Portability

Move workloads across servers with ease without hardware constraints.



Scalability

Scale resources up or down on demand to match workload requirements.

THE RESULT



Higher Efficiency



Lower Cost



Greater Agility



Business Continuity



Virtualization turns one physical machine into many powerful environments—improving efficiency, reducing costs, and accelerating innovation.

One **Host**. Many **Guests**. Infinite Possibilities.

Why Virtualization?

Why Virtualization is the Foundation of Cloud Efficiency

Cloud can exist without Virtualization, but it will be difficult and inefficient.

1. CLOUD WITHOUT VIRTUALIZATION

Possible, but Difficult & Inefficient



One application per physical server



Low resource utilization (much idle capacity)



Hard to scale (manual provisioning)



High cost (over-provisioning)



Slow to respond to demand



Cloud can exist, but delivering elasticity, scalability and cost efficiency becomes very challenging.

2. WHAT CLOUD DELIVERS



"Pay for what you use"

Pay only for the resources you consume.



Infinite availability – use as much you want

Scale on demand and get resources when you need them.

3. MAKING CLOUD PRACTICAL

These notions are practical only if we have:



Lot of Flexibility

Quickly scale up or down, provision in minutes, adapt to demand.



Efficiency in the Back-end

Maximize utilization, minimize waste, optimize resources.

4. THE ENABLER

Virtualization Provides the Efficiency Cloud Needs

VIRTUALIZED ENVIRONMENT

VM

VM

VM

...

HYPERVISOR



Compute



Storage



Network



Physical Machine

- ✓ High resource utilization
- ✓ Rapid provisioning & scalability
- ✓ Better performance
- ✓ Lower cost
- ✓ True cloud experience



Virtualization turns hardware into a flexible, efficient resource pool — making the cloud's promises real.



More Efficiency



Lower Cost



Elastic & Scalable



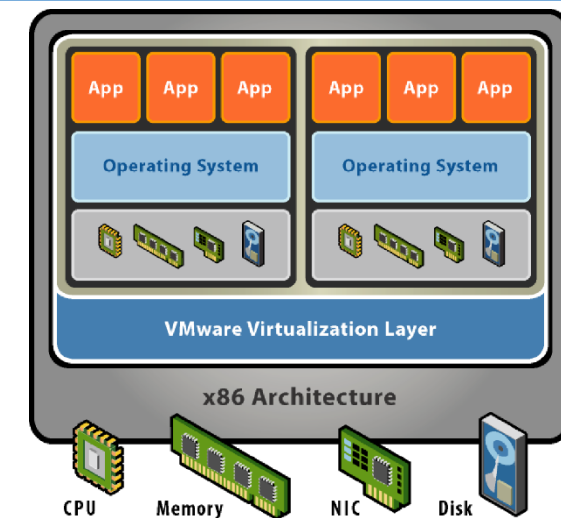
Better Experience



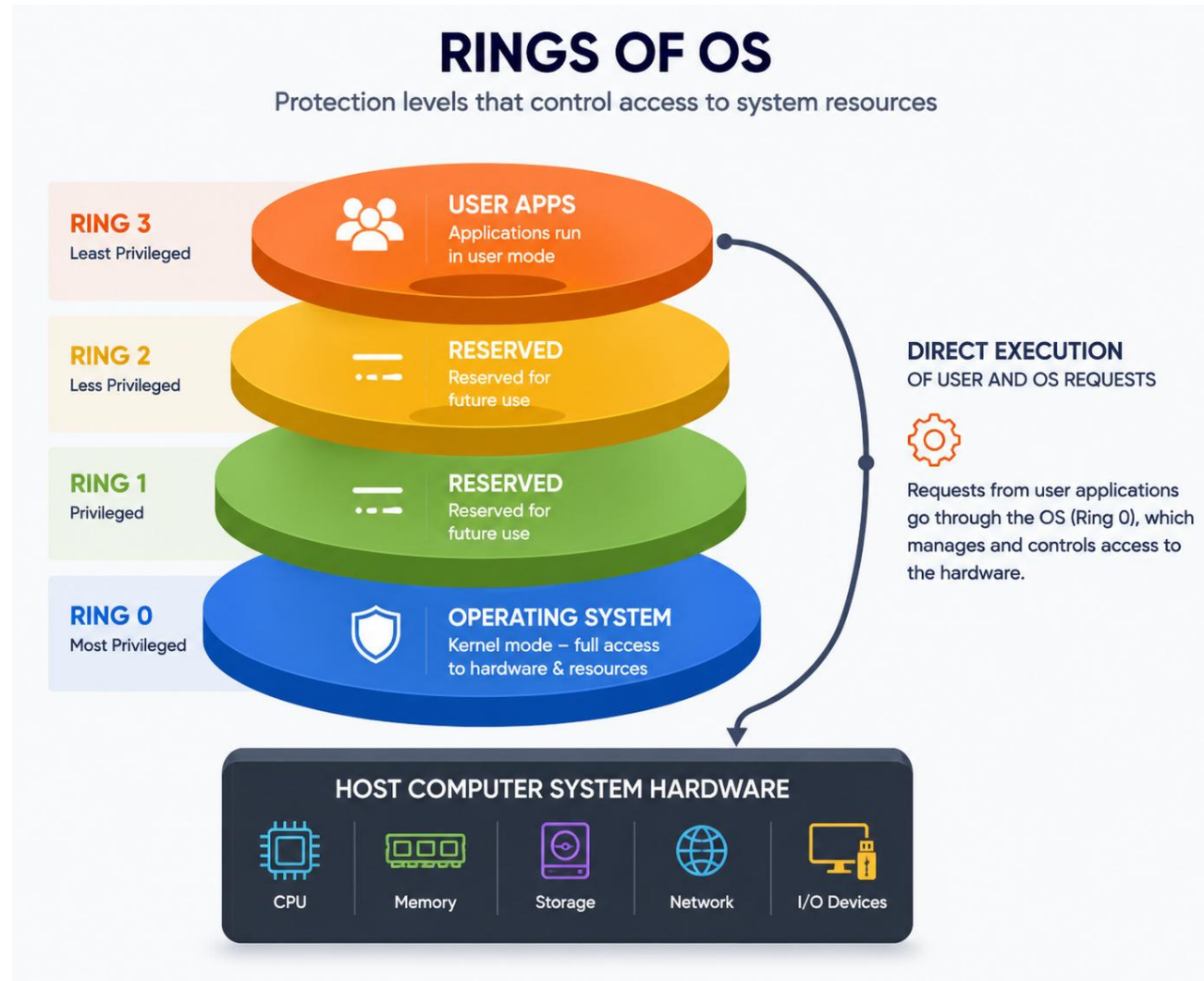
BITS Pilani

Pilani | Dubai | Goa | Hyderabad

The x86 Architecture



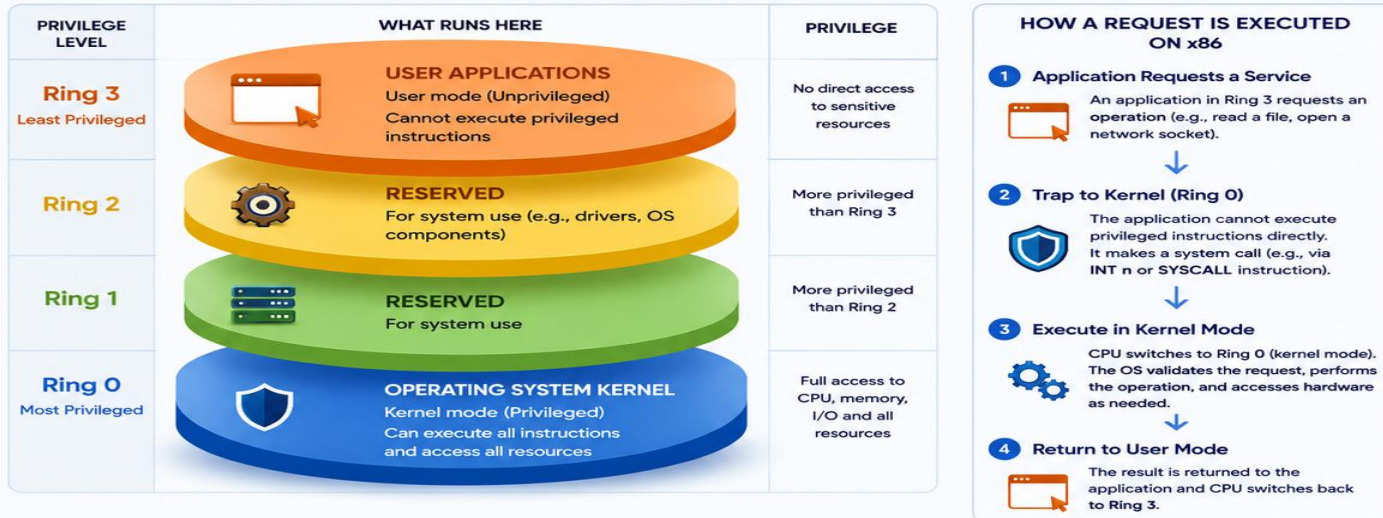
x86 Architecture



x86 Architecture

RINGS OF OS (x86)

How x86 executes requests using privilege levels



PRIVILEGED vs NON-PRIVILEGED

- PRIVILEGED (Run in Ring 0)**
- Can execute all CPU instructions
 - Can access all memory and I/O ports
 - Can perform sensitive operations (e.g., change page tables, manage interrupts, control devices)
 - Used by OS kernel, device drivers, hypervisor, firmware

- NON-PRIVILEGED (Run in Ring 3)**
- Cannot execute privileged instructions
 - Cannot access hardware directly
 - Must request OS services via system calls
 - Used by user applications and most user-space programs

EXAMPLES OF PRIVILEGED INSTRUCTIONS



CLI / STI
(Enable/Disable interrupts)



HLT
(Halt CPU)



IN / OUT
(I/O port access)



MOV to/from CRx
(Control registers)

...
And many more

HOW CPU ENFORCES RINGS



- The CPU maintains a Current Privilege Level (CPL).
- At reset: CPU starts in Ring 0 (kernel mode)
 - When running user apps: CPL = 3
 - System calls or exceptions cause a trap to Ring 0
 - Returning to user mode restores CPL = 3
 - Unauthorized privileged instructions in Ring 3 cause a fault (#GP) — protecting the system

KEY TAKEAWAY



Privilege rings create a protected environment where user applications run safely without risking the stability or security of the system. All critical operations are mediated by the OS kernel in Ring 0.

Analogy:



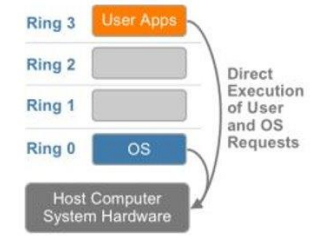
Ring 0 = The King (full authority)
Ring 3 = The Citizen (limited rights)
System calls = Asking the King for a service



Citizens can't change the laws or directly access the treasury — they must submit requests to the King.

- x86 operating systems are built to run directly on bare-metal hardware — they assume they fully own it.
- x86 defines four privilege rings (0–3) to control hardware access.
- To virtualize x86, a hypervisor must sit *underneath* the OS — but the OS still expects Ring 0 for itself. There's no natural "room" left for both.
- Each process running on an x86 system runs in a specific ring, and the ring level determines the level of access the process has to the computer's resources.

X86 Architecture



The x86 architecture uses a system of privilege rings to control access to sensitive areas of the computer's memory and resources. This system is known as Ring Architecture, and there are four privilege levels, or rings, numbered 0 through 3.

- Ring 0:** Also known as the kernel mode, ring 0 is the most privileged level, and it has full access to all of the computer's resources, including memory and I/O devices. This is where the operating system kernel runs.
- Ring 1:** This ring is used for device drivers and other low-level system components. Ring 1 has access to more resources than ring 2 and ring 3, but it still has limited access compared to ring 0.
- Ring 2:** Ring 2 is not used in modern x86 systems, but it was used in the past for system extensions, such as device drivers and system libraries. Ring 2 has even less access to the computer's resources than ring 1.
- Ring 3:** Also known as user mode, ring 3 is the least privileged level, and it is where user applications run. Ring 3 has limited access to the computer's resources, and any access to sensitive areas of the system must be performed through system calls that are handled by the operating system kernel running in ring 0.

Challenges to Virtualization

- The real obstacle isn't just this ring conflict. A hypervisor normally handles privileged instructions by *trapping* them and emulating the result. On x86, a specific set of instructions are **sensitive but not privileged**: they behave differently depending on the current ring, but don't trigger a trap when run outside Ring 0 — they just execute incorrectly, silently, with no error the hypervisor can catch and fix.
- That untrappable-instruction problem is what made x86 virtualization look technically impossible for years — not the ring numbering itself, but the fact that the standard trap-and-emulate technique (which worked fine on other architectures like IBM mainframes) had a real gap here.
- Two fixes eventually emerged: **binary translation** (VMware, 1998) rewrote sensitive instructions in software before execution; **hardware-assisted virtualization** (Intel VT-x / AMD-V, 2006) added a new privilege level below Ring 0 so the CPU itself traps these instructions correctly.



THE AMNESIAC OWNER & THE RESORT – 3 WAYS TO MANAGE THE ILLUSION

THE SETUP

You are the bankrupt owner with amnesia. You believe you still run the resort.

I still own this resort.



The resort estate is the physical hardware.



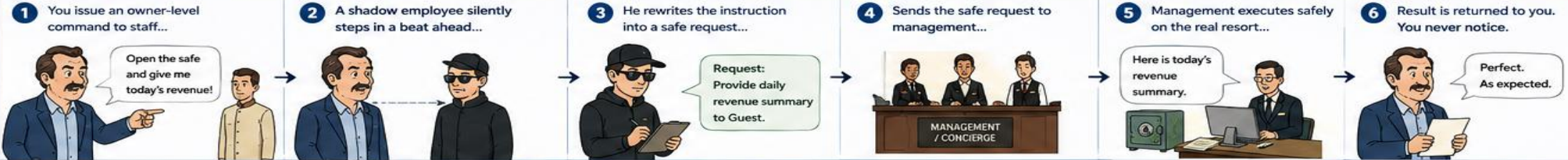
The management / concierge desk is the VMM / hypervisor.



The assistant role changes across scenarios.

1 BINARY TRANSLATION (BT)

You don't know anything.



i The shadow has learned your habits and keeps pre-translated versions ready (caching). But one shadow can barely handle one guest. It's costly and doesn't scale.

2 PARAVIRTUALIZATION (PV)

You are told and retrained.



i No shadow needed because you cooperate. Low cost. But requires the guest to accept the new way. Some guests won't cooperate—those must use BT (shadowing) instead.

3 HARDWARE-ASSISTED VIRTUALIZATION (HW)

You don't know anything.



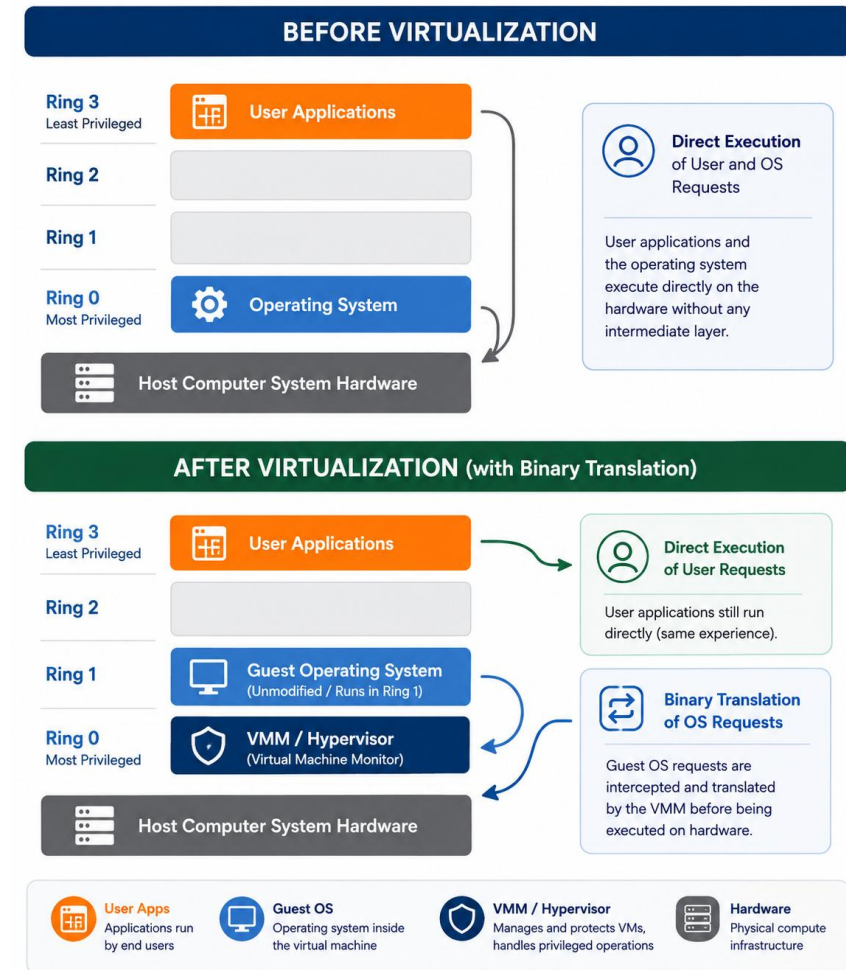
i No shadow. No retraining. The interception is built into the hardware (silicon). Instant, universal, and the most efficient way to maintain the illusion.

HOW THE 3 SCENARIOS COMPARE

	Does the Man Know?	Who Intercepts?	How is it Intercepted?	Cost	Requires Cooperation?	Scalability
1 Binary Translation (BT)	No	Shadow employee (VMM's translator)	Pre-emptive rewriting of anticipated orders (translate before execution, cache & run)	High ●●●	No	Low (one shadow per guest)
2 Paravirtualization (PV)	Yes (told & retrained)	The man himself (cooperating)	Explicit call to management (hypercalls)	Low ●●	Yes	High (once retrained)
3 HW-Assisted Virtualization (HW)	No	The room itself (hardware/silicon)	Automatic, instant reroute on triggering action	Lowest ●	No	Very High (built-in for all guests)

VMware's Solution

- VMware shipped the first commercial solution in 1999 (VMware Workstation 1.0), using binary translation to work around x86's failure to trap ~17 "sensitive but unprivileged" instructions .
- The guest operating system was moved down to Ring 1, a privilege level between the VMM (Ring 0) and applications (Ring 3). This is called ring deprivileging. Binary translation dynamically rewrites the guest kernel's instruction stream at runtime, catching the sensitive instructions that wouldn't otherwise trap, so the guest OS can run safely without knowing it's been demoted.
- While VMware's full virtualization approach using binary translation is the de facto standard today the industry as a whole has not yet agreed on open standards to define and manage virtualization.
- Each company developing virtualization solutions is free to interpret the technical challenges and develop solutions with varying strengths and weaknesses. However, today, KVM's dominance in cloud infrastructure and standards like OVF for VM image portability represent real, if partial, convergence.



Emulation

Emulation is the process by which software reproduces the behavior of hardware — CPU instruction set, memory, and I/O devices — presenting the guest OS with a virtual hardware profile that is completely decoupled from the physical hardware underneath.

Because the guest only interacts with this emulated hardware layer, emulation enables execution across different instruction set architectures (e.g., running an ARM-compiled guest on an x86 host) a capability true virtualization (same-ISA, trap-and-emulate or hardware-assisted) doesn't provide. This cross-ISA independence, not same-ISA compatibility, is emulation's real value proposition.

In a fully emulated environment, both the guest OS and its applications run as ordinary user-mode processes on the host nothing executes natively, so nothing needs elevated privilege on the host. The guest OS remains entirely unaware it isn't running on real hardware.

Emulation

Emulation is implemented via two distinct techniques with very different performance costs:

Interpretation : fetch, decode, and execute each guest instruction one at a time, every time. Simple to implement, computationally expensive, since there's no reuse.

Dynamic binary translation (DBT) : translate a block of guest instructions to equivalent host instructions once, cache the result, and jump directly to it on repeat execution (this is what QEMU's TCG and Apple's Rosetta 2 actually use).

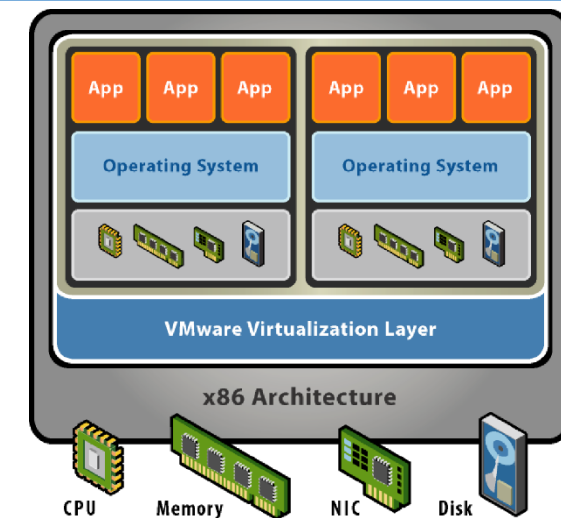
Because even DBT-based emulation carries real overhead compared to native or hardware-assisted execution, full emulation is reserved for cases where ISAs genuinely differ (cross-platform, legacy hardware support, architecture migration) — it is not the mechanism used for standard same-ISA cloud virtualization, where trap-and-emulate or hardware-assisted virtualization (VT-x/AMD-V) dominates instead.



BITS Pilani

Pilani | Dubai | Goa | Hyderabad

Virtualization Approaches



Virtualization Evolution

The Journey to Virtualizing x86

From Impossible to Mainstream

A timeline of key milestones that made x86 virtualization a reality.

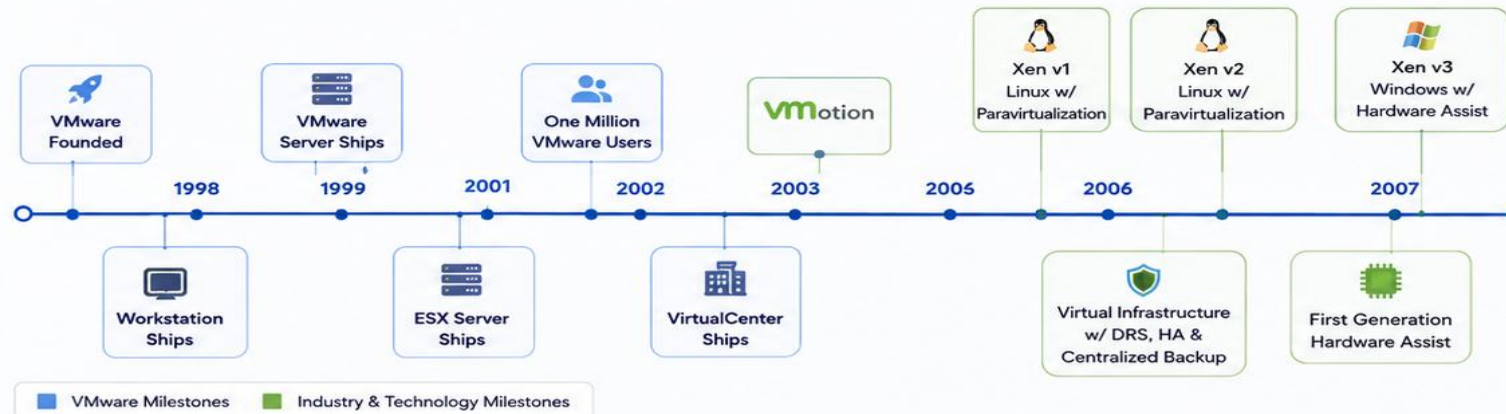


Figure 1 – Summary timeline of x86 virtualization technologies

How We Got Here: Hardware-Assisted Virtualization



Currently, the most commonly used virtualization feature is **hardware-assisted virtualization**.

Hardware-assisted virtualization is a feature built into modern CPUs that provides improved performance and security for virtualization compared to traditional software-based virtualization technologies.

Hardware-assisted virtualization is supported by Intel's VT-x and AMD's AMD-V technology and is used by many popular virtualization platforms, such as VMware, Hyper-V, and Oracle VirtualBox.

This technology allows the virtualization software to run **virtual machines with near-native performance**, as the virtualization software is able to directly access and use the CPU's hardware-assisted virtualization features.



Better Performance
Near-native speed



Stronger Security
Isolated & controlled execution



Broad Support
Used by leading virtualization platforms



Lower Overhead
More efficient virtualization



The Breakthrough Moment

In **1998**, VMware figured out how to virtualize the x86 platform, once thought to be impossible, and created the market for **x86 virtualization**.

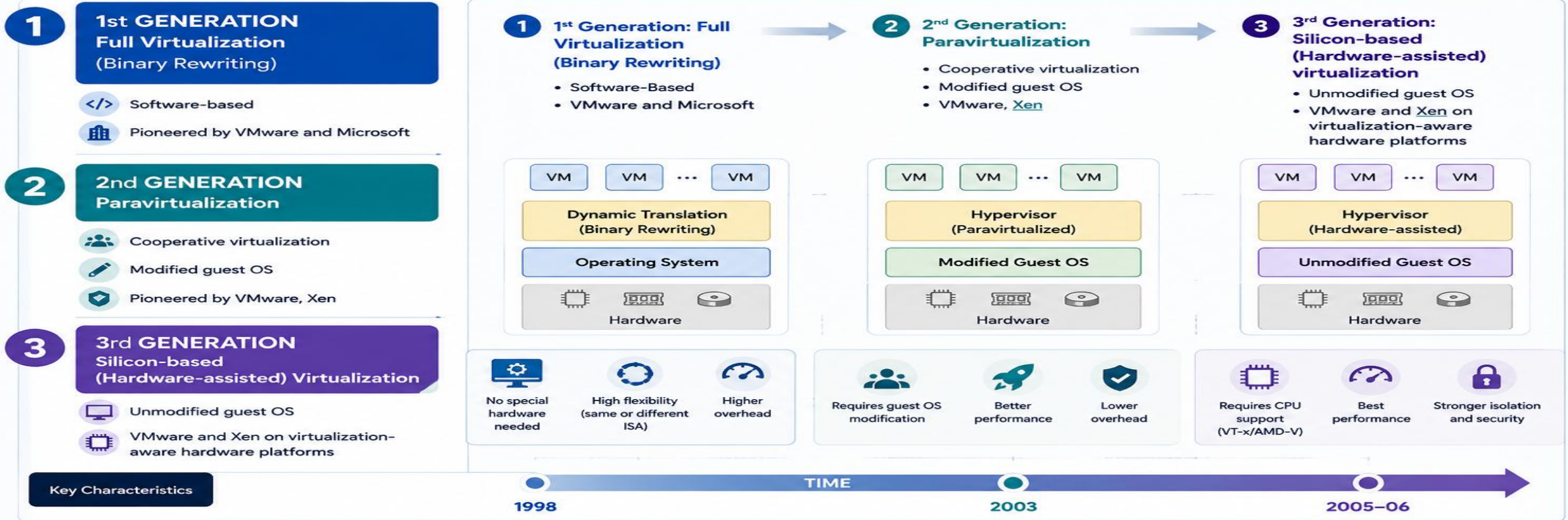
The solution was a combination of **binary translation** and **direct execution** on the processor that allowed multiple guest OSes to run in full isolation on the same computer with readily affordable virtualization overhead.



Introduction

THE EVOLUTION OF VIRTUALIZATION

Three Generations that Transformed Computing



THE BIG PICTURE

From clever software tricks to hardware innovation — virtualization evolved to deliver better performance, stronger security, and the ability to run any OS, at scale.

Binary Translation / Full Virtualization

In its basic form, known as "full virtualization," the hypervisor presents each guest OS with a complete virtual machine:- virtual CPU, memory, and device set that it can run on unmodified, believing it has full hardware ownership. VMware's ESX/Workstation-era hypervisors are the canonical example.

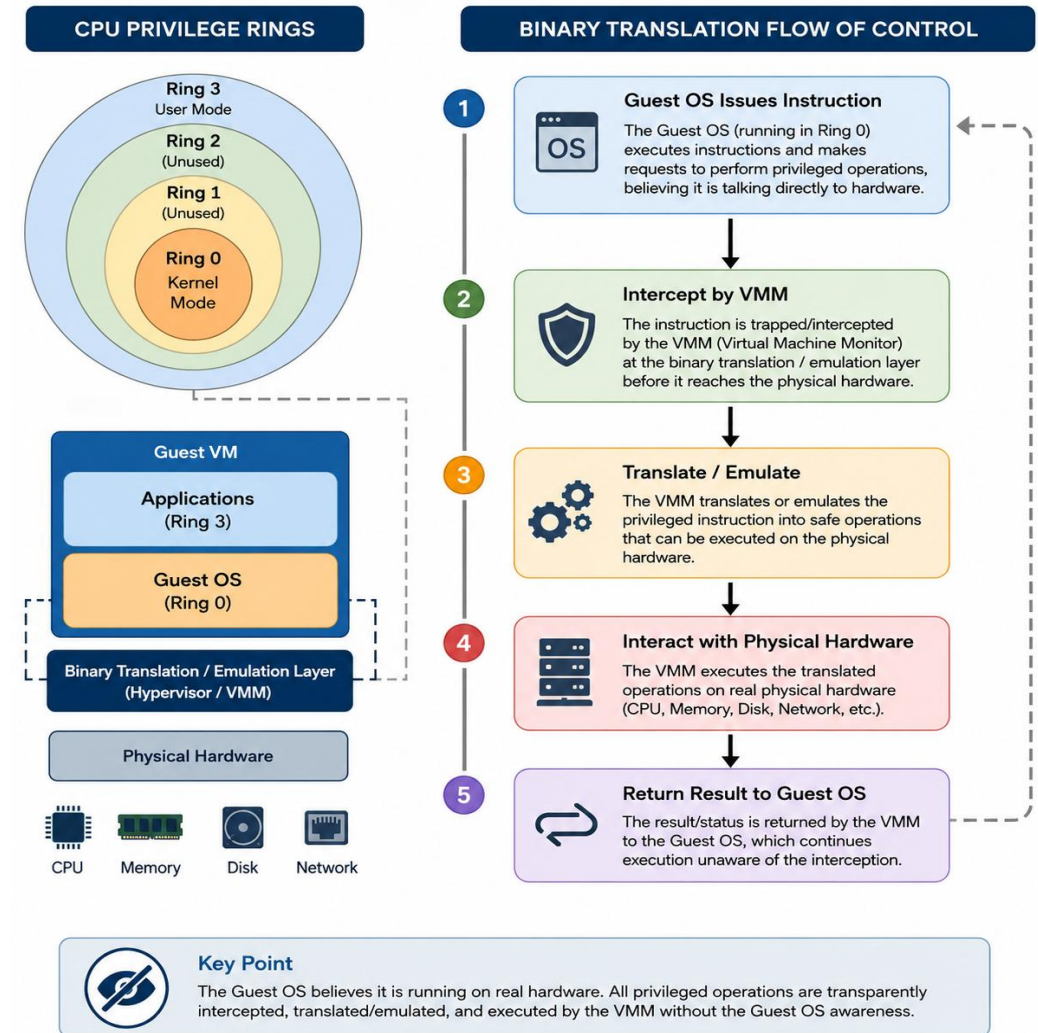
Full virtualization does not provide cross-architecture flexibility meaning, guest and host must share the **same ISA** (e.g., x86 guest on x86 host).

The overhead in this generation came primarily from trapping and translating the small set of sensitive x86 instructions that don't behave correctly outside Ring 0, plus the cost of emulating virtual devices in software.

This was the first viable generation of x86/x64 server virtualization, achieved via dynamic binary translation, and it preceded hardware-assisted virtualization (Intel VT-x / AMD-V, 2005–2006).

Binary Translation / Full Virtualization

Flow of control: the guest OS issues instructions believing it's talking directly to hardware → the hypervisor's translation/emulation layer intercepts and handles them → the layer then interacts with the real physical hardware. The guest OS itself remains unaware it's being intercepted at any point.



Binary Translation / Full Virtualization – Pros/Cons

BINARY TRANSLATION (BT) — PROS & CONS

The software breakthrough that made x86 virtualization possible before hardware support existed

👍 PROS	
	Guest OS runs completely unmodified No source code access, no vendor cooperation needed. This is what let VMware virtualize Windows in 1998 when Microsoft had no reason to help.
	Near-native performance for most instructions Only sensitive/privileged instructions are intercepted — everything else runs directly on real hardware.
	Translated instruction blocks are cached A given code path is translated once and reused, even if executed thousands of times. This is a big reason it wasn't as slow as pure emulation.
	Works on ordinary x86 hardware No special CPU features required, which mattered a lot before VT-x/AMD-V existed in 2005–06.
	Made x86 virtualization viable ~7–8 years earlier Because it doesn't need hardware support, it made x86 virtualization viable roughly 7–8 years before hardware-assisted virtualization existed.

👎 CONS	
	Significant engineering complexity The hypervisor must correctly identify and rewrite every sensitive/non-trapping instruction for the target ISA. Getting this wrong silently breaks guest behavior.
	Translation overhead scales with kernel activity Workloads with heavy kernel activity (frequent context switches, I/O-bound work) see more translation overhead than compute-bound workloads.
	Same-ISA only Unlike emulation, it can't virtualize a guest with a different instruction set than the host.
	Tied to a specific ISA's quirks x86's binary translator doesn't transfer to other architectures without redoing the analysis of which instructions are sensitive-but-not-privileged on that ISA.
	Becomes slower after hardware support matured Once VT-x/AMD-V matured, binary translation became the slower option for most workloads and is now largely a historical/legacy technique.
	Higher hypervisor-side CPU cost Even when running well, the translation engine consumes cycles that hardware-assisted virtualization avoids entirely by trapping in silicon.



THE BIG PICTURE

Binary translation's story isn't "this technique has flaws" — it's **"this was a brilliant, expensive software workaround for a problem that hardware eventually solved properly."** That's why the industry moved on.

Para Virtualization

Paravirtualization, avoids the performance cost of full virtualization/emulation, but modifies the guest OS kernel's architecture-dependent code so the guest cooperates directly with the hypervisor.

This approach required source-level kernel changes, which is why paravirtualization was historically limited to open-source guests (Linux, BSD) rather than closed-source OSes like Windows.

The guest kernel runs deprivileged relative to native execution: Ring 1 on 32-bit x86 (using segment-based protection), or Ring 3 on x86-64 (where long mode doesn't support ring-1/2 segment protection, so isolation relies on separate address spaces instead).

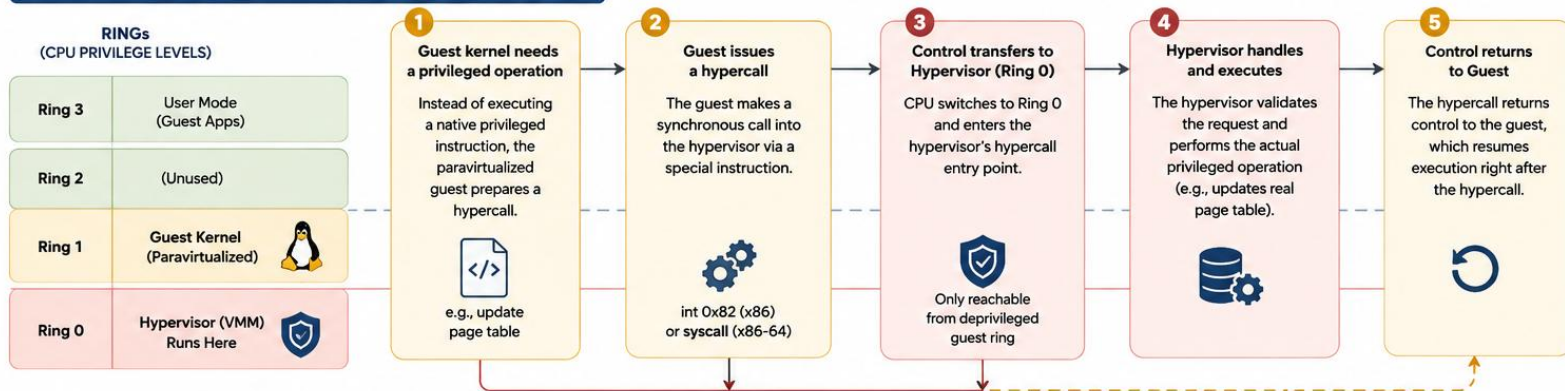
The hypervisor itself occupies Ring 0. Because the guest kernel is modified to know it's virtualized, it doesn't issue the original privileged instructions at all — it replaces them with explicit calls to the hypervisor.

Para Virtualization

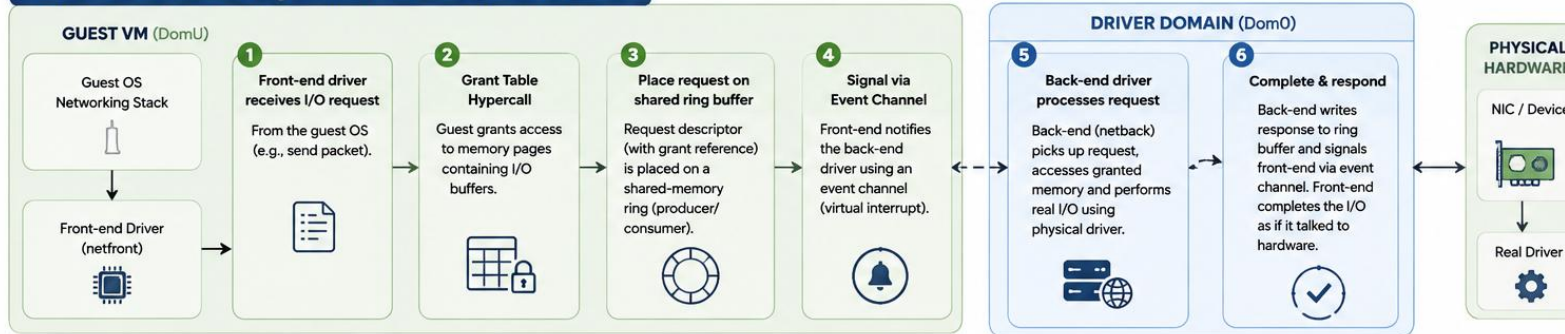
PARAVIRTUALIZATION (PV) – HOW IT WORKS

The guest OS is aware of virtualization and cooperates with the hypervisor via hypercalls and shared mechanisms.

FLOW 1: PRIVILEGED CPU OPERATION (e.g., Page Table Update)



FLOW 2: I/O OPERATION (e.g., Network Send) – SPLIT DRIVER MODEL



LEGEND



Key Takeaway

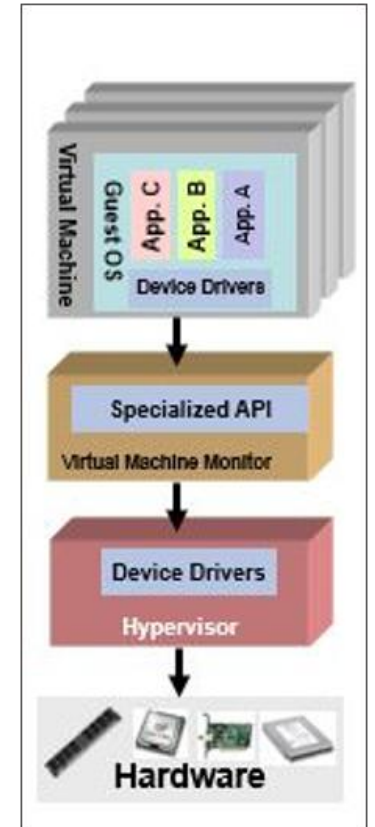
The guest OS cooperates with the hypervisor via hypercalls, shared memory, and event channels to perform privileged operations and I/O efficiently—avoiding traps and emulation.

This means the VMM no longer needs to trap and translate sensitive instructions at runtime (the core overhead of full virtualization/BT) the guest simply asks for privileged operations directly.

The guest OS uses a specialized hypercall API to request privileged operations from the VMM, which executes them against the real hardware on the guest's behalf

Para Virtualization

- In para virtualized VM's there are two different approaches:
- **Recompiling the OS kernel**
 - Para virtualization drivers and APIs must reside in the guest operating system kernel.
 - Modified operating system that includes this specific API, requiring a compiling operating systems to be virtualization aware.
 - Some vendors (such as Novell) have embraced para virtualization and have provided para virtualized OS builds, while other vendors (such as Microsoft) have not.
- **Installing para virtualized drivers**
 - In some operating systems it is not possible to use complete para virtualization, as it requires a specialized version of the operating system
 - To ensure good performance in such environments, para virtualization can be applied for individual devices
 - For example, the instructions generated by network boards or graphical interface cards can be modified before they leave the virtualized machine by using para virtualized drivers



H/W Assisted Virtualization

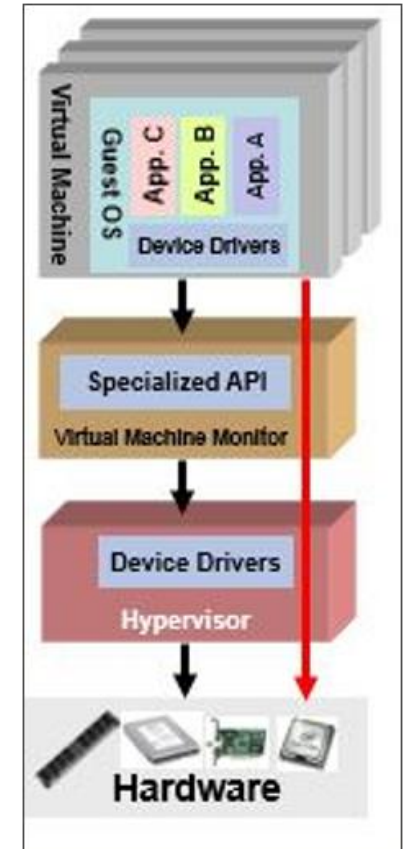
This technique attempts to simplify virtualization because full or paravirtualization is complicated.

Intel and AMD add an additional mode called privilege mode level (some people call it Ring-1) to x86 processors.

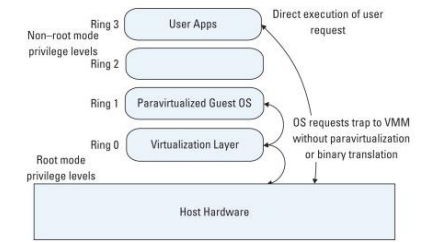
Therefore, operating systems can still run at Ring 0 and the hypervisor can run at Ring -1.

All the privileged and sensitive instructions are trapped in the hypervisor automatically.

This technique removes the difficulty of implementing binary translation of full virtualization. It also lets the operating run without modification unlike para virtualization



H/W Assisted Virtualization



Hardware-assisted virtualization. This term refers to a scenario in which the hardware provides [architectural support](#) for building a [virtual machine manager](#) able to run a guest operating system in complete isolation.

This technique was originally introduced in the IBM System/370. At present, examples of hardware-assisted virtualization are the extensions to the x86-64 bit architecture introduced with *Intel VT* (formerly known as *Vanderpool*) and *AMD V* (formerly known as *Pacifica*).

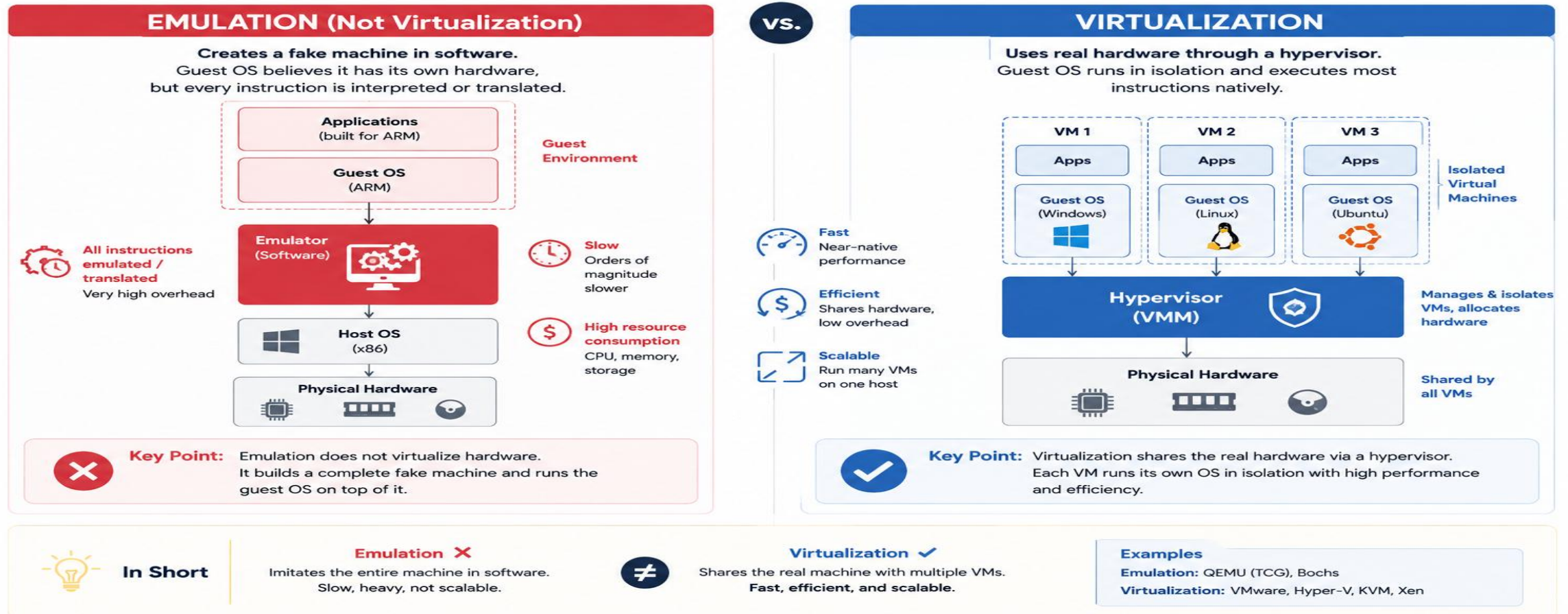
These extensions, which differ between the two vendors, are meant to reduce the performance penalties experienced by emulating x86 hardware with [hypervisors](#). Before the introduction of hardware-assisted virtualization, software emulation of x86 hardware was significantly costly from the performance point of view.

The reason for this is that by design the x86 architecture did not meet the formal requirements introduced by Popek and Goldberg, and early products were using [binary translation](#) to trap some sensitive instructions and provide an emulated version. Products such as VMware Virtual Platform, introduced in 1999 by VMware, which pioneered the field of x86 virtualization, were based on this technique. After 2006, Intel and AMD introduced processor extensions, and a wide range of [virtualization solutions](#) took advantage of them: Kernel-based Virtual Machine (KVM), VirtualBox, Xen, VMware, Hyper-V, Sun xVM, Parallels, and others.

Emulation vs Virtualization

WHY EMULATION IS NOT VIRTUALIZATION

Emulation imitates an entire machine in software. Virtualization shares the real hardware with multiple isolated VMs.





BITS Pilani

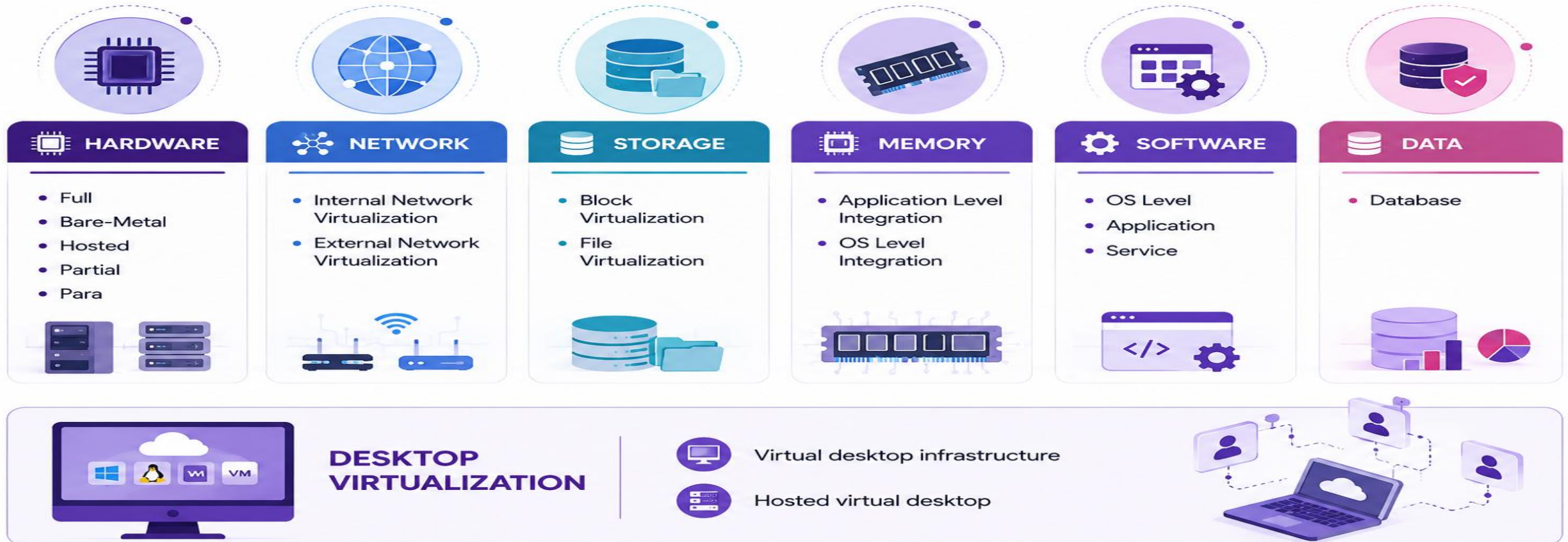
Pilani | Dubai | Goa | Hyderabad

Virtualization Application

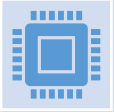
Where do we see Virtualization

VIRTUALIZATION

Virtualization abstracts resources to run multiple workloads efficiently, securely, and independently.



Server Virtualization



Abstracts the physical machine on which the software and operating system is running on and provides an illusion that the software is running on a virtual machine.



Enables Infrastructure as a service model.



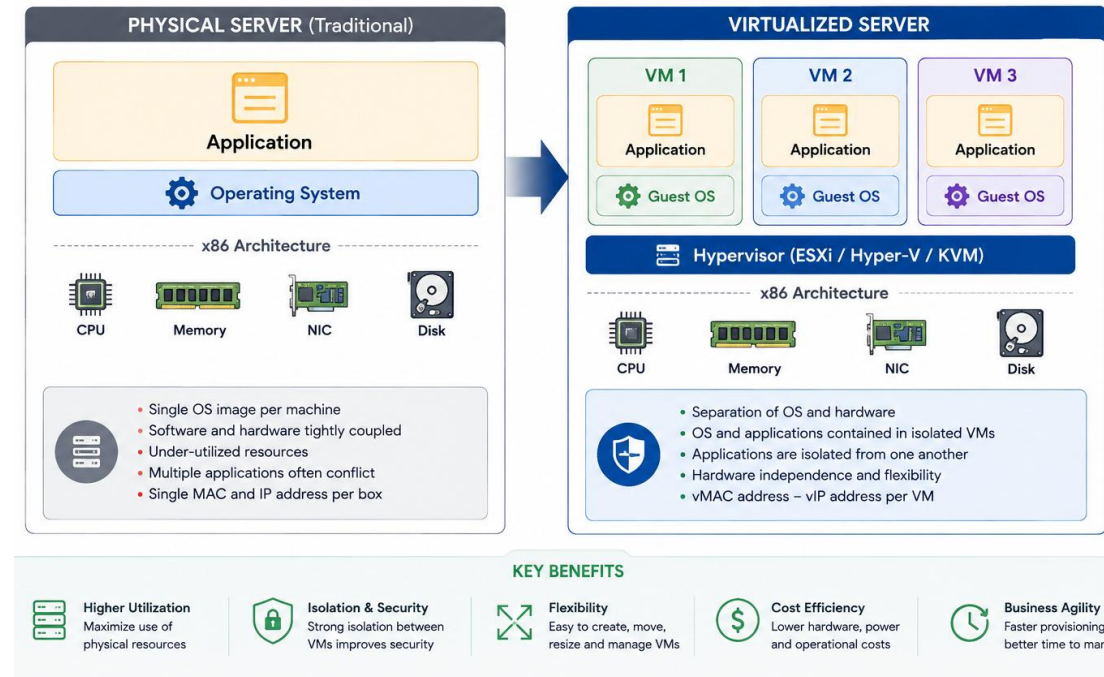
A single physical machine can be used to create several VMs that can run several operating systems independently and simultaneously. VMs are stored as files, so restoring a failed system can be as simple as copying its file onto a new machine.



The hypervisor software enables the creation of a virtual machine (VM) that resembles a physical computer by creating a separate OS environment which is logically isolated from the host server.

SERVER VIRTUALIZATION

Run multiple isolated workloads on a single physical server.



Server Virtualization - Beneits



Partitioning

Run multiple operating systems on one physical machine.

Divide the physical system resources among virtual machines.

One VM does not know the presence of the other.



Management

Failure of one VM does not affect other VMs.

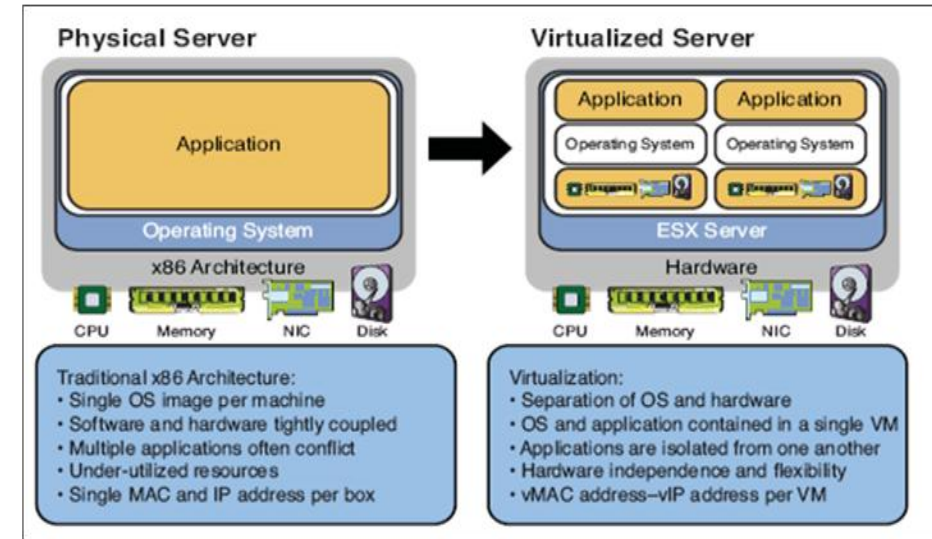
Management agents can be run on each VM separately to determine the individual performance of the VM and the applications that are running on the VM.



Encapsulation

The entire VM state can be saved in a file.

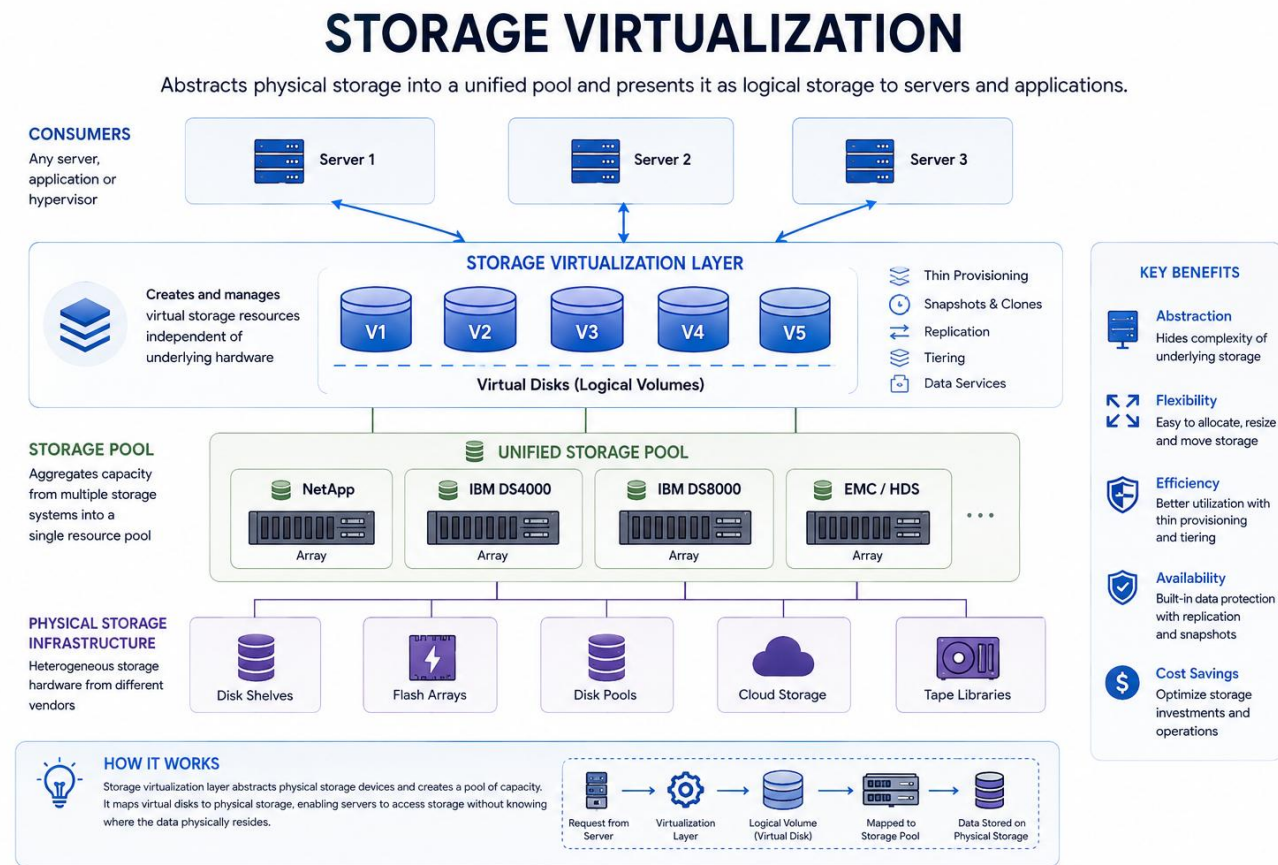
Moving and copying VM information is as easy as copying files.



Server virtualization is a key driving force in reducing the number of physical servers and hence the physical space, cooling, cabling, and capital expenses in any data center consolidation projects

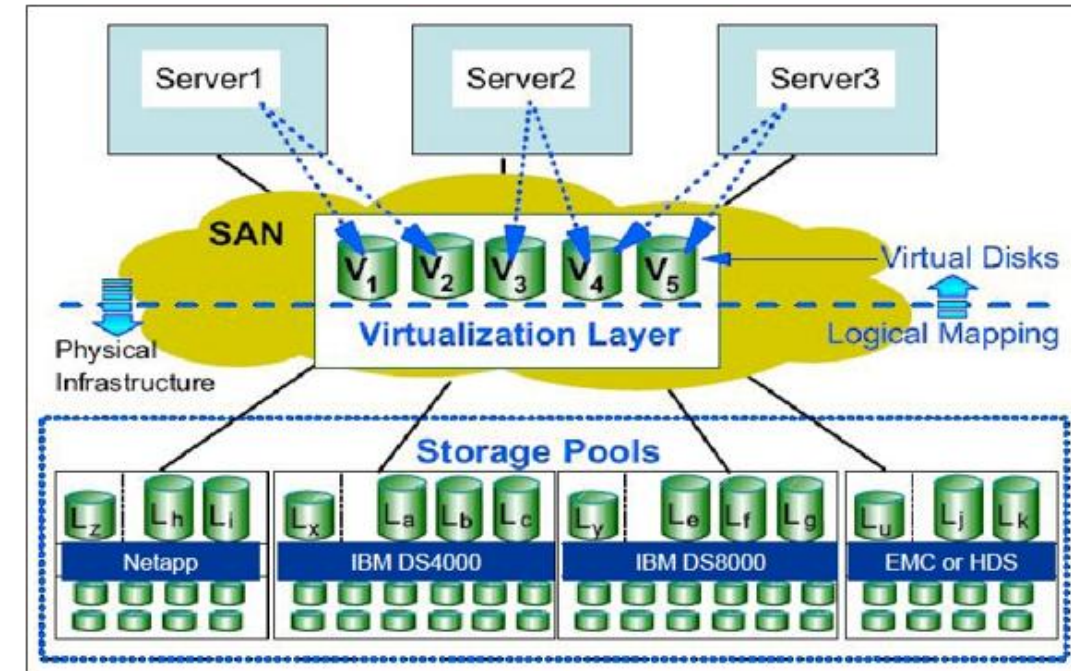
Storage Virtualization

- Storage virtualization refers to providing a logical, abstracted view of physical storage devices.
- It provides a way for many users or applications to access storage without being concerned with where or how that storage is physically located or managed.
- It enables physical storage in an environment to be shared across multiple application servers, and physical devices behind the virtualization layer to be viewed and managed as if they were one large storage pool with no physical boundaries.
- The storage virtualization hides the fact there are separate storage devices in an organization by making all the devices appear as one device.
- Virtualization hides the complex process of where the data needs to be stored and bringing it back and presenting it to the user when it is required.



Storage Virtualization - Benefits

- Typically, the benefits are :-
- **Resource optimization** : Storage virtualization enables you to obtain the storage space on an as-needed basis without any wastage, and it allows organizations to use existing storage assets more efficiently without the need to purchase additional assets.
- **Cost of operation**: Storage virtualization enables adding storage resources without regard to the application, and storage resources can be easily added to the pool by a drag-and-drop method using a management console by the operations people.
- **Increased availability**: Storage virtualization provisions the new storage resources in a minimal amount of time, improving the overall availability of resources
- **Improved performance**

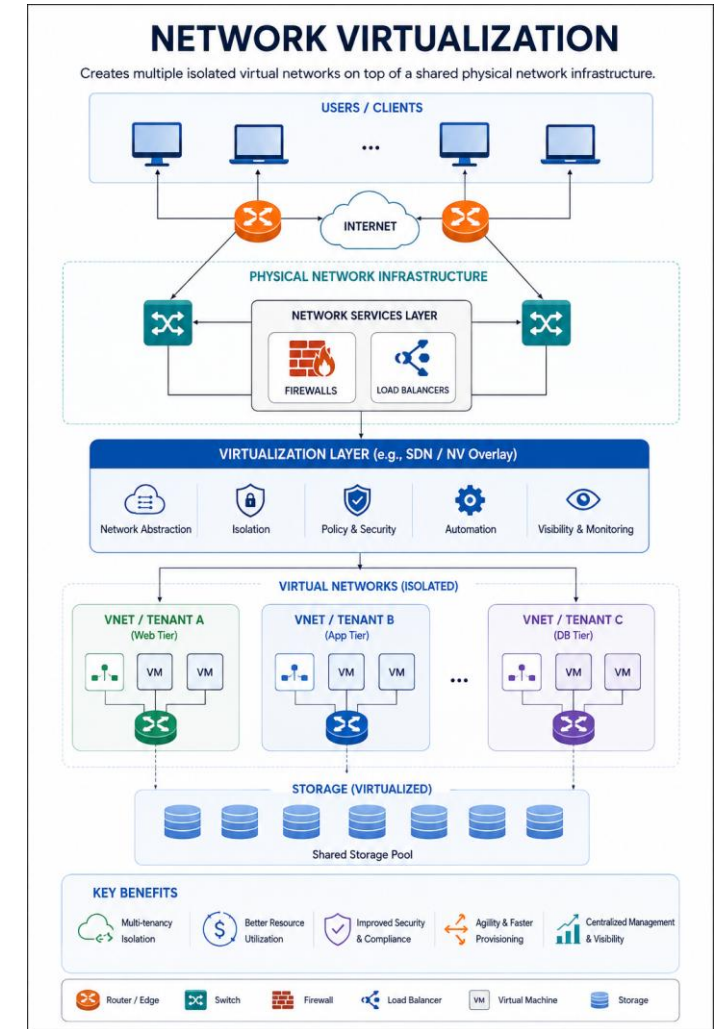


Examples of Storage Virtualization:

- SAN ➔ Storage area Network

Network Virtualization

- Network virtualization might be the most ambiguous virtualization of all virtualization types. Several types of network virtualization exist, as briefly described here:
- A **VLAN** is a simple example of network virtualization. VLANs allow logical segmentation of a LAN into several broadcast domains. VLANs are defined on a switch on a port-by-port basis. That is, you might choose to make ports 1–10 part of VLAN 1 and ports 11–20 part of VLAN 2. There's no need for ports in the same VLAN to be contiguous. Because this is a logical segmentation and not physical, workstations connected to the ports do not have to be located together, and users on different floors in a building or different buildings can be connected together to form a LAN.
- Virtual Routing and Forwarding (VRF), commonly used in Multi-Protocol Label Switching (MPLS) networks, **allows multiple instances of a routing table to coexist within the same router at the same time.**



Memory Virtualization

Beyond CPU virtualization, the next critical component is memory virtualization.

This involves sharing the physical system memory and dynamically allocating it to virtual machines.

Virtual machine memory virtualization is very similar to the virtual memory support provided by modern operating systems.

Applications see a contiguous address space that is not necessarily tied to the underlying physical memory in the system.

The operating system keeps mappings of virtual page numbers to physical page numbers stored in page tables. All modern x86 CPUs include a memory management unit (MMU) and a translation lookaside buffer (TLB) to optimize virtual memory performance

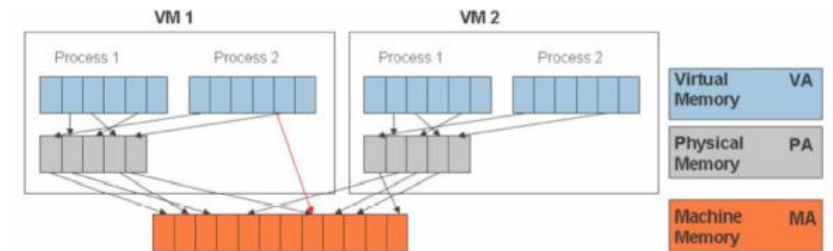


Figure 8 – Memory Virtualization

Device Virtualization

Device virtualization presents each VM with a virtualized version of a peripheral device — network adapter, disk controller, video adapter, USB controller, and so on

The guest OS interacts with what it believes is a real device; the hypervisor is the one actually talking to physical hardware underneath.

As one can access remotely, you are able to work from any location and on any PC. It provides a lot of flexibility for employees to work from home or on the go.

It also protects confidential data from being lost or stolen by keeping it safe on central servers.

This involves managing the routing of I/O requests between virtual devices and the shared physical hardware.

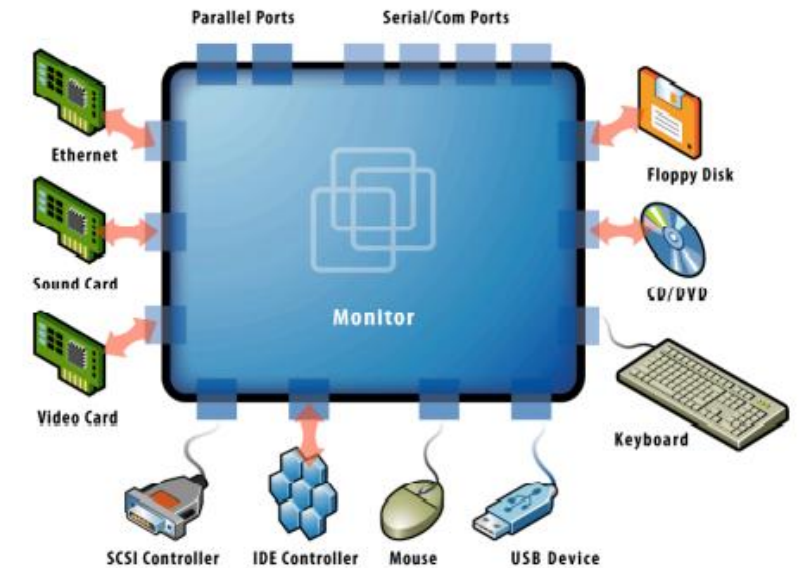
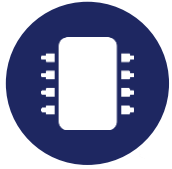


Figure 9 – Device and I/O virtualization

CPU Overcommitment — Concept



The hypervisor can allocate more vCPUs across VMs than physical cores/threads (pCPUs) exist on the host. This works because CPU time is time-sliced — the scheduler hands out short slices of physical core time the same way an OS multiplexes processes onto cores.

Overcommit Ratio

$$\text{Total vCPUs} \div \text{Total pCPUs}$$



CPU Ready Time

Time a vCPU is ready to run but waiting for a physical core to free up. Rising CPU Ready (commonly flagged above ~5%) signals the host is oversubscribed beyond what current workloads can tolerate.



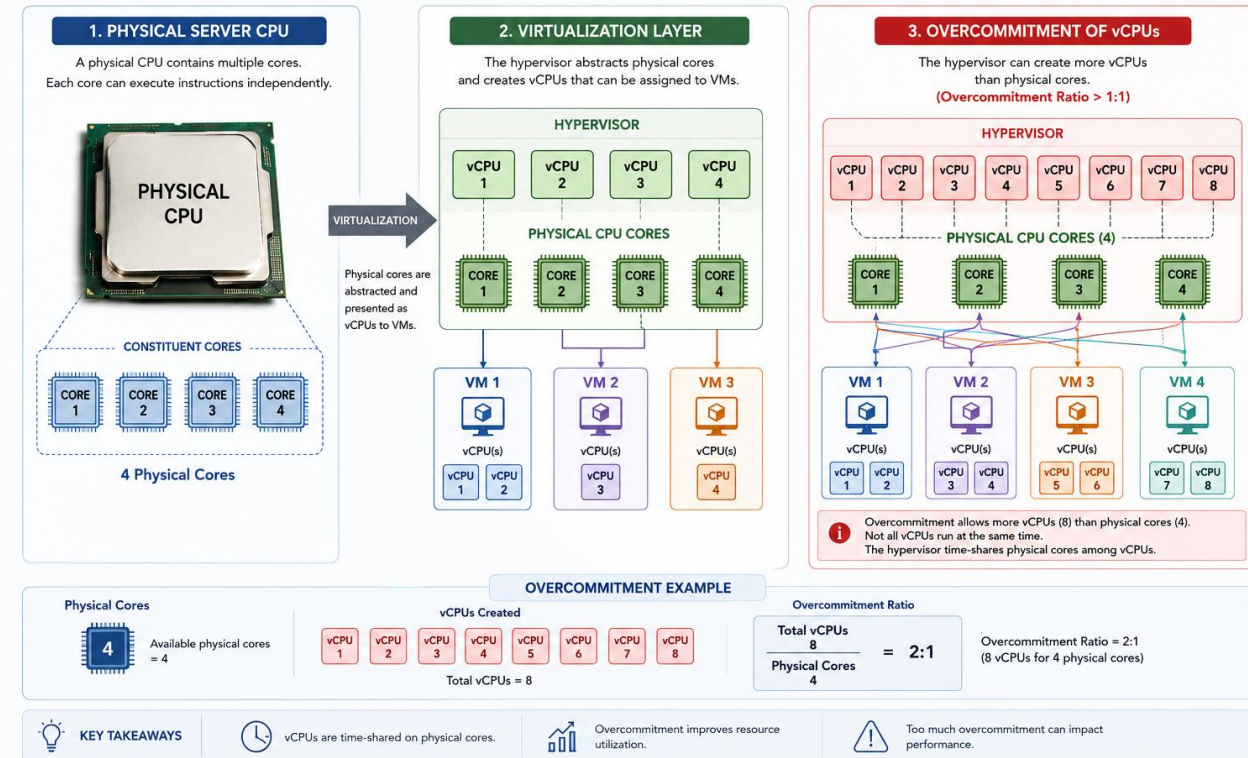
Rough Guidance (workload-dependent)

1:1 — latency-sensitive (DBs, real-time)

3:1–5:1 — workable for mixed workloads

>8:1 — high risk by most practitioners

FROM PHYSICAL CPU CORES TO vCPUs IN VIRTUAL MACHINES WITH OVERCOMMITMENT



CPU Overcommitment — Example

Setup: A host has 4 physical cores (pCPUs), no hyperthreading. 6 VMs are deployed, each configured with 2 vCPUs.

1

Total vCPUs allocated

$6 \text{ VMs} \times 2 \text{ vCPUs} = 12 \text{ vCPUs}$

2

Overcommit ratio

$12 \text{ vCPUs} \div 4 \text{ pCPUs} = 3:1$

3

Is this safe?

3:1 sits at the edge of the “safe zone” (3:1–5:1) — acceptable, but worth watching



Step 4 — the subtlety: If 3 of the 6 VMs simultaneously peg both vCPUs to 100%, that's $3 \times 2 = 6$ vCPUs of must-run-now demand against only 4 pCPUs — effective contention of $6:4 = 1.5:1$ for that busy subset, even though the whole-host ratio looks fine at 3:1. The aggregate ratio is a planning heuristic, not a guarantee.

Memory Overcommitment — Concept



Memory can't be time-sliced the way CPU cycles can — a page of RAM is either allocated to a VM or it isn't, at any given instant. The hypervisor can't schedule its way out of a shortfall; it must actively reclaim or deduplicate memory.

Overcommit Ratio

Configured Memory ÷ Physical RAM

Reclamation techniques — least to most invasive:

1

Page Sharing (TPS / KSM)

Merges identical memory pages. Since vSphere 6.0, inter-VM sharing is disabled by default for security — only intra-VM sharing is on unless explicitly re-enabled.

2

Ballooning

A guest-side driver reclaims memory from the guest's own processes — the guest OS decides what to page out.

3

Compression

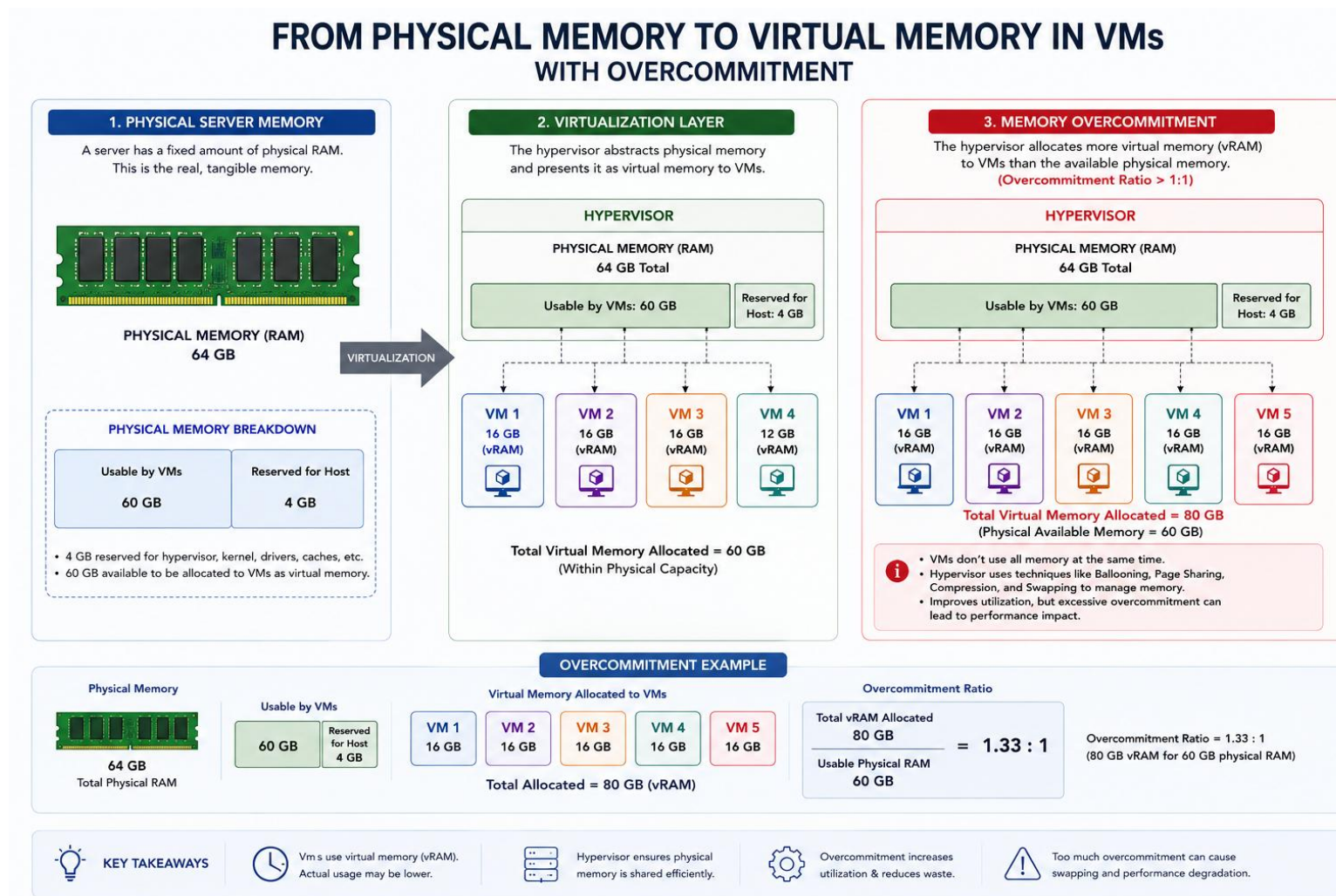
Hypervisor compresses pages instead of writing to disk — slower than RAM, much faster than disk.

4

Host-level Swapping

Last resort — pages swapped to disk without guest cooperation. Disk is ~100,000× slower than RAM.

Memory Overcommitment — Concept



Memory Overcommitment — Example

Setup: A host has 128 GB of physical RAM. 20 VMs are deployed, each configured with 8 GB.

1 Total configured memory $20 \times 8 \text{ GB} = 160 \text{ GB}$

2 Overcommit ratio $160 \text{ GB} \div 128 \text{ GB} = 1.25:1$

3 But ratio alone isn't the risk signal Suppose actual peak active working-set demand = 140 GB



Step 4 — the shortfall: 140 GB of active demand against 128 GB physical is a 12 GB shortfall. The host tries page sharing and ballooning first; if those can't reclaim fast enough, it falls back to compression, and ultimately host-level swapping — at which point affected VMs show sudden, sharp latency spikes.

Virtualization Advantages

VIRTUALIZATION ADVANTAGES



01 SECURITY

Isolates workloads with different security requirements in separate virtual machines.

Choose the right OS and tools for each environment.

A security issue in one VM does not impact others.



02 RELIABILITY & AVAILABILITY

A failure in one VM does not affect other VMs.

High availability and disaster recovery are easier to achieve.



03 COST EFFICIENCY

Consolidate workloads to run more VMs on fewer physical servers.

Reduces hardware, power, space, operations, and software licensing costs.

Typical savings: 29% – 64%.



04 ADAPTABILITY TO WORKLOAD VARIATIONS

Easily adjust resources (CPU, memory, storage) and priorities.

Move workloads between VMs dynamically for optimal performance.



05 LOAD BALANCING

VM state is fully encapsulated by the VMM.

Easily migrate VMs across hosts to balance load and improve performance.



06 LEGACY APPLICATION SUPPORT

Run legacy applications and older OS versions as guest OS within a VM.

Reduces migration effort and cost while preserving business continuity.



KEY TAKEAWAY:

Virtualization delivers secure, reliable, cost-effective and agile IT—enabling organizations to innovate faster while optimizing resources.

Virtualization Comparison

Evolution of Virtualization Approaches

Four key approaches that made virtualization possible on x86

DIMENSION	 1. EMULATION	 2. BINARY TRANSLATION	 3. PARAVIRTUALIZATION	 4. HARDWARE-ASSISTED VIRTUALIZATION
 Core idea	Software simulates an entirely different CPU/ISA instruction-by-instruction.	Hypervisor translates sensitive instructions at runtime; everything else runs natively.	Guest OS kernel is modified to call the hypervisor directly instead of executing privileged instructions.	CPU adds a new privilege mode below Ring 0 so the hypervisor traps sensitive instructions in hardware.
 Guest OS modified?	✗ No	✗ No	✓ Yes — kernel source must be changed	✗ No
 Guest ISA vs. host ISA	Can differ (e.g., ARM guest on x86 host)	Must match	Must match	Must match
 How privileged/sensitive instructions are handled	Every instruction interpreted or JIT-translated , none run natively.	Trapped and rewritten dynamically at runtime by the hypervisor.	Replaced at compile-time with hypercalls to the hypervisor.	Trapped automatically by the CPU itself, no software translation needed.
 Non-privileged instructions	Also interpreted/translated (nothing runs natively).	Run directly on real hardware ("direct execution").	Run directly on real hardware.	Run directly on real hardware.
 Performance overhead	Highest — often 10–100x slower (Likely, workload-dependent)	Moderate — better than emulation, worse than the two below.	Low — near-native for CPU-bound work.	Lowest achievable in software-based eras; comparable to or better than paravirtualization once mature. (Early VT-x/AMD-V slower; improved with EPT/RVI)
 Real examples	QEMU (pure emulation mode), Bochs, DOSBox	VMware Workstation/ESX (1998–2006 era), early VirtualBox	Xen (with modified Linux guest), User-Mode Linux	KVM, modern ESXi, Hyper-V, Xen in HVM mode
 Requires special CPU support?	✗ No	✗ No	✗ No	✓ Yes — Intel VT-x or AMD-V
 Key limitation	Too slow for production workloads; used for cross-architecture dev/test, not data centers.	Complex hypervisor engineering; translation overhead scales with how instruction-heavy the workload is.	Can't run an unmodified/closed-source OS (historically blocked Windows guests until source cooperation existed).	Needs hardware from ~2006 onward ; won't run on older CPUs.
 Era of prominence in your deck's timeline	Not covered by CS3 — pre-dates and sits outside its 3-generation model	1st 1 st generation (1998, VMware)	2nd 2 nd generation (2003, Xen)	3rd 3 rd generation (2005–06, VT-x/AMD-V)

Points to Note

- **Software licensing**

One of the most significant virtualization-related issues to be aware of is software licensing. Virtualization makes it easy to create new servers, but each VM requires its own separate software license.

Organizations using expensive licensed applications could end up paying large amounts in license fees if they do not control their **server sprawl**.

- **IT training**

IT staff used to dealing with physical systems will need a certain amount **of training in virtualization**. Such training is essential to enable the staff to debug and troubleshoot issues in the virtual environment, to secure and manage VMs, and to effectively plan for capacity.

- **Hardware investment**

Server virtualization is most effective when **powerful physical machines** are used to host several VMs. This means that organizations that have existing not-so-powerful hardware might still need to make upfront investments in acquiring new physical servers to harvest the benefits of virtualization.

Q & A.....





BITS Pilani

Pilani | Dubai | Goa | Hyderabad

Credits

*Hwang, Kai; Dongarra, Jack; Fox, Geoffrey C.. Distributed and Cloud Computing: From Parallel Processing to the Internet of Things (Kindle Locations 3532-3533). Elsevier Science. Kindle Edition.